DSA03-NATURAL LANGUAGE PROCESSING
ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: **04-11-2026**

NAME: T. SHIVA NARAYANA
REG NO: 192472027

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.

- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Reevaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks
Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated twoway equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.

- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.

- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.

- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.

- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example "patient ku fever irundhuchu, paracetamol 500mg kuduthen". At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, Englishonly morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

Project objective

Build a finite-state language-processing layer that converts noisy clinical free text into structured entities and searchable canonical forms. The system combines regular expressions, finite automata, a finite-state morphological analyser, a Porter stemmer and a robustness layer for OCR errors and Romanised Tamil-English code-mixing.

Overall system requirements

Requirement	Design target	How it will be evidenced
Corpus	≥500 de-identified/synthetic records; ≥25,000 tokens; ≥4 record types	Corpus manifest, generator/collection protocol, token count
Structured patterns	12 entity types	Regex inventory, positive and negative tests
Automata	≥4 automata; ≥6 states each; one NFA example	Diagrams, transition tables, subset construction
Morphology	≥300 stems; ≥20 inflectional and ≥15 derivational affixes	Lexicon and affix inventory files
Evaluation	≥40 queries; ≥200 annotated forms	Relevance judgements and gold annotation sheet
Day-4 challenge	OCR noise + Romanised regional language	Noisy test set and revised pipeline

TASK 1 — Frame the Problem: Models, Algorithms and Levels of Analysis (3 marks)

1.1 Levels of linguistic analysis

Level	Meaning	Hospital example	Required? / reason
Phonology	Sound systems and pronunciation.	Different pronunciations of a drug name may lead to spelling variants in speech transcription.	Not central for typed records; becomes relevant if speech input is added.
Morphology	Internal word structure and word forms.	fracture, fractures, fractured, fracturing; nephrectomy, nephrectomies.	Essential: query expansion and canonicalisation.
Syntax	How words combine into phrases and sentences.	'500 mg twice daily' versus 'twice daily 500 mg'.	Partly required: helps identify dosage and relations between values.
Semantics	Meaning of words and phrases.	renal is related to kidney; 'negative' in a lab report has clinical meaning.	Required for synonym and concept linking; simple regex cannot provide it.
Pragmatics	Meaning depending on speaker, purpose and context.	'Rule out fracture' does not mean the patient has a fracture.	Important for safe retrieval; avoid treating every mention as confirmed diagnosis.
Discourse	Meaning across sentences or a whole document.	A note says 'suspected infection'; a later sentence says 'culture negative'.	Useful for document-level interpretation, but not needed for basic tokenisation.

Conclusion: the minimum finite-state search layer must handle tokenisation, structured entities and morphology. A safe clinical search system also needs limited semantic and contextual handling so that negation, uncertainty and historical mentions are not misrepresented. Full phonological and discourse understanding can be deferred unless speech or document-level reasoning is in scope.

1.2 Three model families

Model family	Best hospital task	Why it fits	Why the other families are weaker here
State machines (DFA/NFA/FST)	Validate fixed formats such as patient IDs, dates and dosage strings; analyse inflectional forms.	They recognise regular patterns with predictable transitions and efficient runtime.	A probabilistic model may accept malformed IDs; a hand-written rule list is less systematic for many paths.
Formal rule systems	Apply spelling rules, abbreviation expansion, date canonicalisation and morphology.	Rules are explicit, auditable and reproducible.	A state machine alone does not conveniently express contextual rewrite rules; probability is unnecessary for deterministic normalisation.
Probabilistic models	Rank ambiguous search results, identify likely OCR corrections or classify uncertain abbreviations.	They estimate likelihood from context and tolerate variation.	A deterministic automaton cannot rank alternatives; hand rules become brittle for noisy and ambiguous text.

The recommended design is hybrid: deterministic rules for high-confidence structure, finite-state morphology for known word forms, and probabilistic/contextual ranking only where ambiguity or noise requires it.

1.3 Ambiguity in the corpus

Ambiguity	Example	Why it is ambiguous	Handling
Lexical	cold	Could mean low temperature, common cold, or cold sensation.	Use surrounding words and section type.
Morphological	saw	Could be past tense of see or a noun.	Return multiple analyses; rank by context.
Segmentation	500mg	Could be one number-unit token or two tokens.	Maximal-munch pattern treats it as one measurement token.
Abbreviation	MS	May mean multiple sclerosis, morphine sulfate, or medical student.	Expand only with section/context evidence; preserve original.
OCR	rn vs m; O vs 0	Character recognition creates lookalike forms.	Weighted edit distance and field constraints.
Code-mixing	patient-ku fever irundhuchu	English clinical terms embedded in Romanised Tamil.	Token-level language identification and bilingual lexicon.

Ambiguity is central because one surface string can have multiple meanings or analyses, while one concept can have many surface forms. A safe system should preserve alternatives and evidence instead of forcing a single interpretation too early.

1.4 Search in the unit and resource costs

- Automata perform state-space search: at each input symbol, the machine follows an allowed transition. A DFA has one active state; an NFA may have a set of active states.
- Regex engines may use backtracking to explore alternative matches. Poorly designed nested quantifiers can cause excessive time; prefer bounded, anchored patterns for clinical fields.
- FST lookup traverses lexical and surface paths. Runtime depends on input length and transducer size; compiled finite-state lookup is typically efficient.
- Resources: time (matching and analysis), memory (automata/lexicon/index), annotated data (gold labels and relevance judgements), and expert effort (clinical lexicon, abbreviation review, pattern maintenance).

1.5 Labelled pipeline

INPUT
Clinical records: discharge summaries / prescriptions / operation notes / nursing notes
|
v
[1. Ingestion & de-identification] Formalism: file parsing + privacy rules
|
v
[2. OCR / text extraction] Formalism: OCR engine (noisy text channel)
|
v
[3. Tokenisation] Formalism: regex + maximal munch
|
v
[4. Structured entity recognition] Formalism: anchored regex + DFA/NFA validators
|
v
[5. Normalisation] Formalism: rewrite rules / lookup tables
|
v
[6. Morphological analysis] Formalism: lexicon FST + morphotactic rules
|
v
[7. Fallback stemming] Formalism: Porter rule algorithm
|
v
[8. Semantic/context checks] Formalism: terminology map + negation/context rules
|
v
[9. Indexing and retrieval] Formalism: inverted index + query expansion/ranking

|
v
OUTPUT: searchable canonical tokens, structured entities, analyses and ranked clinical records

The output must retain both the original text and the normalised form. The original is needed for auditability and for clinicians to inspect the exact source wording.

TASK 2 — Basic Regular Expression Patterns (4 marks)

2.1 Basic operators with clinical examples

Operator	Meaning	Clinical example
Concatenation	A followed by B.	[A-Z]{2}[0-9]{6}: two letters followed by six digits.
Disjunction |	Either alternative.	mg|mcg: either unit.
Kleene star *	Zero or more repetitions.	[0-9]*: optional digit sequence (often use carefully).
Plus +	One or more repetitions.	[0-9]+: one or more digits.
Optional ?	Zero or one occurrence.	-?[0-9]+: optional minus sign.
Character class	One character from a set.	[0-9] matches one digit.
Range	Continuous character range.	[A-Z] matches uppercase English letter.
Negated class	Any character except listed characters.	[^\s,;]+ matches a non-space/non-comma token.
Anchors ^ $	Start and end of string.	^[A-Z]{2}[0-9]{6}$ validates the whole identifier.
Counters {m,n}	Bounded repetition.	[0-9]{2}/[0-9]{2}/[0-9]{4}

2.2 Twelve clinical entity patterns

The patterns below use Python's re syntax. They are deliberately conservative examples. Real hospital deployment must validate local ID formats, date ranges, units and drug catalogue names with clinical informatics staff.

Entity	Illustrative Python regex	Must match / must not match
Patient identifier	\\b(?:MRN|PID|UHID)\\s*[:#-]?\\s*[A-Z0-9]{6,12}\\b	Matches labelled identifiers such as MRN: AB12345 6. Rejects unlabelled arbitrary numbers and IDs shorter than 6 characters.

Admission date	\\b(?:Admission\\s*Date|Admitted\\s*on)\\s*[:=-]?\\s*(?:\\d{4}-\\d{2}-\\d{2}|\\d{2}[/-]\\d{2}[/-]\\d{4})\\b	Matches a labelled date in ISO or day/month/year style. Rejects impossible dates after calendar validation.
Discharge date	\\b(?:Discharge\\s*Date|Discharged\\s*on)\\s*[:=-]?\\s*(?:\\d{4}-\\d{2}-\\d{2}|\\d{2}[/-]\\d{2}[/-]\\d{4})\\b	Matches labelled discharge date. Rejects dates with missing year or invalid calendar day.
Drug name + strength	\\b(?:paracetamol|amoxicillin|metformin|ibuprofen|atorvastatin|insulin)\\s+\\d+(?:\\.\\d+)?\\s*(?:mg|mcg|g|ml|IU)\\b	Matches listed drug plus numeric strength/unit. Rejects unknown drug names or strength without a unit; expand using formulary lexicon.
Dosage frequency	\\b(?:once|twice|thrice)\\s+daily\\b|\\b(?:OD|BD|BID|TDS|TID|QID|QHS|PRN)\\b	Matches common frequency phrases/codes. Rejects arbitrary letter sequences

		such as ABC.
Laboratory value + unit	\\b(?:Hb|WBC|RBC|platelets|glucose|creatinine|sodium|potassium)\\s*[:=]?\\s*\\d+(?:\\.\\d+)?\\s*(?:g/dL|mg/dL|mmol/L|10\\^9/L|mEq/L)\\b	Matches named lab analyte, value and approved unit. Rejects values without an analyte or unit.
Blood pressure	\\b(?:BP\\s*[:=]?\\s*)?(?:[6-9]\\d|1[0-9]{2}|2[0-4]\\d)\\s*/\\s(?:[3-9]\\d|1[01]\\d)\\s*mmHg\\b	Matches plausible systolic/diastolic range followed by mmHg. Rejects e.g. 20/200 mmHg.
Temperature	\\b(?:Temp(?:erature)?\\s*[:=]?\\s*)?(?:3[4-9]|4[0-2])(?:\\.\\d)?\\s*°?C\\b	Matches 34–42°C with optional decimal. Rejects 25°C and 50°C as likely invalid clinical temperatures.
ICD code	\\b[A-TV-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b	Matches common ICD-10 style code shape. Rejects wrong syntax; code validity still needs official

		ICD lookup.
Ward and bed	\\bWard\\s*[A-Z0-9-]{1,8}\\s*[,;]+Bed\\s*[A-Z0-9-]{1,8}\\b	Matches labelled ward and bed. Rejects ward-only or bed-only values.
Contact number	\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b	Matches Indian 10-digit mobile number, optional +91. Rejects landlines and numbers starting 0–5.
Age	\\b(?:Age\\s*[:=]?\\s*)?(?:0|[1-9][0-9]{0,2})\\s*(?:years?|yrs?|months?|days?)\\b	Matches age with unit. Rejects negative ages and values above 999; plausibility check is separate.
Symptom duration	\\b(?:for|since)\\s+\\d+\\s*(?:hours?|hrs?|days?|weeks?|months?|years?)\\b	Matches duration phrases. Rejects duration without a unit.

The planning brief says 'at least 12' and lists 13 items when admission and discharge dates are counted separately. This report covers all 13, which satisfies the minimum.

2.3 Greedy and non-greedy matching

```
text = "Give 500 mg twice daily and 250 mg once daily."
re.findall(r".*mg", text)       # greedy: may span from first character to the LAST 'mg'
```

```
re.findall(r".*?mg", text)      # non-greedy: shortest span ending at the FIRST possible 'mg'
```

For dosage extraction, neither broad expression is safe. Use a bounded pattern such as r'\b\d+(?:\.\d+)?\s*mg\b' to return exactly '500 mg' and '250 mg'. Greedy quantifiers expand as far as possible; non-greedy quantifiers expand only as far as needed for the next part of the pattern.

2.4 Precedence and grouping

```
# Intended: either "BP:" followed by a value, OR "Blood pressure:" followed by a value
r"^(?:BP|Blood pressure):\s*\d{2,3}/\d{2,3}$"

# Incorrect grouping: anchors apply only to one branch in many regex engines
r"^BP|Blood pressure:\s*\d{2,3}/\d{2,3}$"
```

The incorrect expression means (^BP) OR (Blood pressure: digits/digits$). It can silently accept strings beginning with BP even when the remainder is invalid. Group alternatives and anchor the entire expression.

2.5 Evaluation and error analysis

For every pattern, construct a gold-labelled test set with positive examples and hard negatives. Count TP (correct matches), FP (non-entities matched) and FN (entities missed).

Pattern family	Typical false positive cause	Typical false negative cause
Patient ID	A serial number mistaken for an ID.	OCR confuses O/0 or ID prefix omitted.
Dates	A historical date mistaken for admission date.	Date written as '5 Nov 24' not covered.
Drug + strength	Drug name in unrelated text.	Misspelling or brand name absent from lexicon.
Frequency	BD used as an abbreviation with another meaning.	'every 8 hours' not included.
Lab value	A number next to wrong unit.	Alternative unit or analyte spelling.
BP / temperature	Out-of-context numeric pair/value.	No unit or OCR corruption.
ICD	Syntactically shaped but invalid code.	Local extension or code version not covered.
Ward/bed	Ward code alone resembles combined pattern.	Different order: Bed 12, Ward A.
Contact / age / duration	Phone number interpreted as ID; age without unit.	Words or regional language used instead of digits.

Report per-pattern precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2PR/(P+R). Do not claim results until the labelled corpus has been processed.

TASK 3 — Regular Expressions for Tokenisation and Normalisation (4 marks)

3.1 Token definition and tokenizer

A token is a maximal sequence that is meaningful for indexing: words, abbreviations, identifiers, decimal measurements, units and dates. Hyphenated clinical terms remain one token (e.g., 'follow-up'); dotted abbreviations remain one token (e.g., 'N.B.'); decimals remain one token (e.g., '7.5'); and attached measurements remain one token (e.g., '500mg').

```
TOKEN_RE = re.compile(
    r"(?i)"
```

```
    r"(?:[A-Z]{2,}\.){1,}[A-Z]?\.?"            # e.g. N.B., U.S.
    r"|(?:\d{4}-\d{2}-\d{2}|\d{2}[/-]\d{2}[/-]\d{4})"  # dates
    r"|(?:\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml|mmol/L|g/dL|mmHg|°C|IU))"
    r"|(?:[A-Za-z]+(?:[-'][A-Za-z]+)+)"        # hyphenated / apostrophe words
    r"|(?:[A-Za-z]+)"
    r"|(?:\d+(?:\.\d+)?)"
    r"|(?:[^\w\s])"
)
```

Maximal munch means select the longest valid token at the current position. In implementation, order longer and more specific alternatives before shorter ones.

3.2 Maximal munch versus whitespace split

Input	Naive whitespace split	Maximal-munch tokenisation
'BP 120/80 mmHg'	BP | 120/80 | mmHg	BP | 120/80 | mmHg (BP value can be a structured span)
'Take 500mg twice-daily'	Take | 500mg | twice-daily	Take | 500mg | twice-daily
'N.B. HbA1c was 7.5%'	N.B. | HbA1c | was | 7.5%	N.B. | HbA1c | was | 7.5 | %
'2024-05-06 follow-up'	2024-05-06 | follow-up	2024-05-06 | follow-up
'O2 saturation 98%'	O2 | saturation | 98%	O2 | saturation | 98 | %

3.3 Normalisation rules and 30 abbreviations

Normalisation must be field-aware. Preserve original token, canonical token and rule ID. Expand an abbreviation only when context makes the meaning sufficiently clear.

Abbreviation	Canonical expansion
BP	blood pressure
HR	heart rate
RR	respiratory rate
Temp	temperature
SOB	shortness of breath
DOE	dyspnoea on exertion
N/V	nausea and vomiting
C/O	complains of
H/O	history of
K/C/O	known case of
DM	diabetes mellitus
HTN	hypertension
CAD	coronary artery disease
CKD	chronic kidney disease
COPD	chronic obstructive pulmonary disease
CVA	cerebrovascular accident
UTI	urinary tract infection
MI	myocardial infarction
ECG	electrocardiogram
CXR	chest X-ray
OD	once daily
BD	twice daily
TDS	three times daily

QID	four times daily
PRN	as needed
PO	by mouth
IV	intravenous
IM	intramuscular
NPO	nothing by mouth
stat	immediately

Date canonicalisation: convert valid dates to ISO YYYY-MM-DD after resolving local date order. Unit canonicalisation: map equivalent spellings to a controlled unit string, e.g. 'µg' and 'mcg' → 'mcg'. Case folding: convert general text to lowercase, but preserve original casing for identifiers and gene/protein symbols where case is meaningful.

3.4 Full before-and-after example

Synthetic nursing note (original):

N.B. Mr AB123456 admitted on 05/06/2024.
C/O fever for 2 days. BP 120/80 mmHg, Temp 38.2 C.
Rx: Paracetamol 500mg PO BD. Review on 2024-06-07.

Normalised representation (assuming DD/MM/YYYY locale):

n.b. mr ab123456 admitted on 2024-06-05.
complains of fever for 2 days. blood pressure 120/80 mmhg,
temperature 38.2 °c. prescription: paracetamol 500 mg by mouth
twice daily. review on 2024-06-07.

The original string is retained in the database; the normalised string is an additional search field.

3.5 Irreversible normalisation decisions

- Lowercasing destroys original case. Acceptable for ordinary prose search, not for case-sensitive identifiers or gene symbols.
- Abbreviation expansion may discard the original shorthand and may choose the wrong sense. Keep both original and expanded forms.
- Unit conversion (e.g., mg to g) can introduce rounding and lose the originally documented precision. Preserve original value/unit alongside converted value.
- Removing punctuation can collapse distinctions such as decimal points or initials. Never strip punctuation before parsing measurements, dates and abbreviations.

3.6 A requirement regex cannot express

A requirement to match arbitrarily nested balanced parentheses, such as '(pain (left side (after surgery)))', is not regular. A finite automaton has only finite memory and cannot count an unbounded number of opening brackets to ensure the same number of closing brackets. A stack-based context-free parser is needed for arbitrary nesting.

TASK 4 — Finite State Automata: Design, Non-determinism and Equivalence (5 marks)

4.1 Formal definition

A finite automaton is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators

For readability, transitions below use character classes as labels. Each validator can be expanded into individual character transitions in code. Each DFA has at least six logical states when prefix, body, separator and accepting/rejecting progress states are counted.

Validator	State path (compressed)	States	Acceptance condition
Patient ID (MRN: AB123456)	q0→q1 M; q1→q2 R; q2→q3 N; q3→q4 ':'/space; q4→q5 A–Z; q5→q6 A–Z; q6→q7 digit×6; q7 accept	q0, q1, q2, q3, q4, q5, q6, q7	Accept only exact labelled format; reject short or extra characters.
ISO date YYYY-MM-DD	q0–q3: four digits; q4 '-'; q5–q6: two digits; q7 '-'; q8–q9: two digits; q10 accept	q0…q10	Shape validation; calendar validation checks month/day.
BP systolic/diastolic	q0–q2 systolic digits; q3 '/'; q4–q6 diastolic digits; q7 'mmHg'; q8 accept	q0…q8	Numeric range checks after format recognition.
Contact number +91 optional	q0 optional '+'; q1 '9'; q2 '1'; q3 optional separator; q4 first digit 6–9; q5–q13 nine more digits; q14 accept	q0…q14	Accepts 10-digit Indian mobile number with optional country prefix.

4.3 Transition table example: ISO date DFA

Current state	Input class	Next state	Accepting?
q0	digit	q1	No
q1	digit	q2	No
q2	digit	q3	No
q3	digit	q4	No
q4	-	q5	No
q5	digit	q6	No
q6	digit	q7	No
q7	-	q8	No
q8	digit	q9	No
q9	digit	q10	Yes
Any	other	dead	No

The date DFA validates shape. A second semantic calendar check rejects 2024-19-45. This separation keeps the automaton finite and the date logic explicit.

4.4 NFA with epsilon transitions: optional country prefix

Use a phone-number format that optionally contains the +91 country prefix and an optional separator. The NFA branches at the start: one path consumes a local 10-digit number; the other consumes +91 and then the same local number.

```
NFA states: {s, p, c, d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, f}
Start: s     Final: {f}
Transitions:
s --ε--> d1              # local number path
```

```
s --ε--> p              # country-code path
p --'+'--> c
c --'9'--> c1
c1 --'1'--> c2
c2 --ε--> d1
d1 --[6-9]--> d2
d2 --[0-9]--> d3
...
d10 --[0-9]--> f
```

Accepted trace for '9876543210': ε-closure(s) = {s,p,d1}; consume 9 → {d2}; consume next eight digits through d10; final digit reaches f, so accept. For '5876543210', first digit 5 has no transition from d1, so the active state set becomes empty and the string is rejected.

A complete implementation uses a 10-digit loop state rather than listing each digit position; the expanded DFA can still be represented with position states for an assignment diagram.

4.5 Subset construction and minimisation

NFA state-set	Input '+'	Input '9'	Input '1'	Input digit 6–9	Input digit 0–9	Accept?
A = ε-closure({s}) = {s,p,d1}	{c}	{d2}	∅	{d2}	∅	No
B = {c}	∅	{c1}	∅	∅	∅	No
C = {c1}	∅	∅	{c2}	∅	∅	No
D = {c2,d1}	∅	{d2}	∅	{d2}	∅	No
E = {d2}	∅	∅	∅	∅	{d3}	No
...	...	...	...	...	...	...
F = {f}	∅	∅	∅	∅	∅	Yes

For the final submission, expand the omitted digit-position rows for the 10-digit suffix and include the dead state. The subset construction algorithm is: start with ε-closure({s}); for each unprocessed subset and each alphabet symbol compute ε-closure(move(S,a)); add any new subset; mark subsets containing an NFA final state as accepting.

Minimisation groups equivalent DFA states with the same future acceptance behaviour. In this phone example, the local-number and country-prefix branches merge after the prefix is consumed. State counts must be computed from the fully expanded transition table; do not report a minimised count from the compressed sketch.

4.6 Thompson construction and regex equivalence

Example regex: ^[0-9]{4}-[0-9]{2}-[0-9]{2}$. Thompson construction builds an NFA fragment for each digit class, concatenates fragments with ε links, and inserts literal hyphen transitions. The resulting NFA accepts exactly strings with four digits, hyphen, two digits, hyphen, two digits.

Conversely, state elimination derives a regular expression from an automaton: add a new start and final state; eliminate intermediate states one at a time; when eliminating state k, update each edge i→j with $R_{ij} \cup R_{ik}(R_{kk})^*R_{kj}$. The resulting expression for the date-shape DFA is [0-9]{4}-[0-9]{2}-[0-9]{2}. Equivalence is established because both describe the same accepted strings.

4.7 Pumping lemma and non-regularity

Pumping lemma: if L is regular, there exists p such that every string s in L with $|s| \geq p$ can be written s=xyz where $|xy| \leq p$, $|y|>0$, and $xy^iz \in L$ for every $i \geq 0$.

Let $L = \{ ('^n)'^n : n \geq 0 \}$, the language of strings with equal numbers of opening parentheses followed by closing parentheses. Assume L is regular with pumping length p. Choose s='('*p + ')'*p. Since $|xy| \leq p$, y consists only of '(' symbols. Pump i=0; the result has fewer opening brackets than closing brackets and is not in L. Contradiction. Therefore L is not regular. This explains why regex/FSA alone cannot validate arbitrary nested balanced brackets.

TASK 5 — Inflectional Morphology (3 marks)

5.1 Definitions

Term	Definition	Example
Morpheme	Smallest unit carrying meaning or grammatical function.	fracture + s
Stem	Form to which an affix is attached.	fracture
Affix	Bound morpheme attached to a stem.	-s, -ed, -ing
Inflection	Changes grammatical form but normally preserves word class and core meaning.	fracture → fractures
Paradigm	Set of grammatical forms of a lexeme.	treat, treats, treated, treating

Inflection usually preserves category and is highly productive. Derivation may change category or meaning and is more restricted. Example: treat (verb) → treated (verb, inflection); treat (verb) → treatment (noun, derivation).

5.2 Inflectional paradigms and clinical lexemes

Paradigm	Examples
Noun number	fracture/fractures; lesion/lesions; infection/infections; kidney/kidneys; dose/doses
Verb tense/aspect	treat/treats/treated/treating; admit/admits/admitted/admitting; bleed/bleeds/bled/bleeding
Subject agreement	patient has; patients have; he receives; they receive
Adjective comparison	mild/milder/mildest; severe/more severe/most severe (analytic comparison common)

Lexeme	Base	Plural / 3sg	Past	Progressive / other
fracture	fracture	fractures	fractured	fracturing
lesion	lesion	lesions	—	—
treat	treat	treats	treated	treating
admit	admit	admits	admitted	admitting
study	study	studies	studied	studying
monitor	monitor	monitors	monitored	monitoring

5.3 English spelling rules

Rule	Implementation idea	Counter-example / over-generation risk
e-deletion	Drop final silent e before -ing: make→making.	see→seeing; do not delete e from every word ending e.

Consonant doubling	Double final consonant for short stressed CVC stems before -ed/-ing: admit→admitted.	visit→visited; stress and syllable conditions matter.
y-to-i	Consonant+y becomes i before -es/-ed: study→studies/studied.	play→plays/played because y follows vowel.
sibilant -es	Add -es after s, x, z, ch, sh: wash→washes.	gas has gases but some words have lexical exceptions.

5.4 Irregular forms

Lemma	Irregular forms	Reason
be	am, is, are, was, were, been, being	Suppletion / stem alternation
have	has, had, having	Irregular agreement and past
go	goes, went, gone	Suppletion
see	saw, seen	Vowel alternation
take	took, taken	Vowel alternation
give	gave, given	Vowel alternation
write	wrote, written	Vowel/consonant alternation
child	children	Irregular plural

Irregular forms must be stored as lexical mappings because a general productive suffix rule cannot derive them reliably.

5.5 Measuring inflectional impact

Type-to-lemma ratio = number of distinct surface word types / number of distinct lemmas. A lower ratio indicates that many surface forms map to fewer base lexemes. Compute the ratio using the annotated corpus and FST analyses.

```
type_to_lemma_ratio = len(surface_types) / len(unique_lemmas)
inflection_query_failure_rate = inflection_only_missed_queries / total_queries
```

Report the exact numerator and denominator. Example reporting template: 8,400 surface types / 6,100 lemmas = 1.38. This is only an example calculation; replace it with corpus counts.

TASK 6 — Derivational Morphology (4 marks)

6.1 Derivation and affix inventory

Derivation creates a new lexeme and may change word class or meaning. Clinical vocabulary uses many Greek and Latin combining forms. The following inventory is a starter resource; retain only forms attested in the chosen corpus and document frequency.

Affix	Type	Typical meaning / category effect	Example
-itis	Suffix	inflammation; noun	nephritis
-ectomy	Suffix	surgical removal; noun	nephrectomy
-ology	Suffix	study/field; noun	neurology
-osis	Suffix	condition/process; noun	fibrosis
-aemia	Suffix	blood condition; noun	anaemia
-ation	Suffix	action/result; noun	inflammation (related family)
-able	Suffix	capable of; adjective	treatable
-ive	Suffix	having property; adjective	obstructive

-al	Suffix	relating to; adjective	renal
-ic	Suffix	relating to; adjective	gastric
-ous	Suffix	characterised by; adjective	infectious (related -ious)
-ment	Suffix	result/process; noun	treatment
-er	Suffix	agent/person; noun	examiner
-less	Suffix	without; adjective	painless
-ly	Suffix	manner; adverb	clinically
nephro-	Prefix/combining form	kidney	nephrology
cardio-	Prefix/combining form	heart	cardiology
hyper-	Prefix	excessive/high	hypertension
hypo-	Prefix	low/under	hypoglycaemia
re-	Prefix	again/back	reassessment
un-	Prefix	not/reversal	untreated

6.2 Derivational families

Root/family	Derivational family	Attachment order / note
nephr/kidney	nephro- → nephrology; nephro- + -ectomy → nephrectomy; renal	Combining form + suffix; renal is a related Latin form, not a simple suffix chain.
cardi/heart	cardio- → cardiology; cardiac; cardiomegaly	Prefix/combining form precedes suffix.
infect	infect → infection → infectious; uninfected	infect + -ion; infection + -ous; un- + infected.
treat	treat → treatment → treatable; untreated	-ment/-able attach to verb; un- attaches to adjective/participle.
operate	operate → operation → operative; postoperative	post- + operative; operation is a nominalisation.
fracture	fracture → fractures; fractured; refracture	Inflectional plural differs from derivational re-.

Impossible or ill-formed orders include *-itisnephro (suffix before root), *treat-un-ed (un- inserted between stem and inflection), and *ectomy-nephro. Affix order is constrained by morphotactics and lexical convention.

6.3 Semantic non-compositionality and search risk

- Heartburn is not literally combustion of the heart; aggressive stemming to heart may create misleading matches.
- Cold can mean a viral illness or low temperature; matching cold to every temperature record can be clinically noisy.
- Renal and kidney are synonyms but have different surface roots; suffix stripping alone will not connect them.
- Nephrectomy means surgical removal of a kidney; matching every nephro- term to nephrectomy would be too broad.

Derivational relatives should not all be conflated. Conflation helps for renal/kidney and nephrectomy/nephrectomies when the clinician searches for the same procedure. It can mislead for infection/infectious (condition versus property) and treat/treatment (action versus result). Use concept links with relation labels rather than destructive stemming.

6.4 Productivity measurement

For each affix, count distinct derived types and total tokens. A productive affix appears with many different stems and continues to form new words; a fossilised affix occurs in a small fixed set.

Affix	Types (to calculate)	Tokens (to calculate)	Expected classification
-itis	count distinct stems + affix	total occurrences	Productive in medical terminology
-ectomy	count distinct procedure roots	total occurrences	Moderately productive / domain-specific
-ology	count distinct field names	total occurrences	Productive
-osis	count distinct condition names	total occurrences	Productive but semantically varied
-aemia	count distinct blood-condition names	total occurrences	Domain-specific
-able	count distinct adjective stems	total occurrences	Highly productive
hyper- / hypo-	count distinct stems	total occurrences	Productive
un- / re-	count distinct stems	total occurrences	Productive in general English

Do not invent counts. Generate them from the corpus by matching the affix inventory against lexicon entries and manually checking false segmentations.

TASK 7 — Finite-State Morphological Parsing (5 marks)

7.1 Finite-state transducer definition

A finite-state transducer (FST) is a finite-state machine whose transitions consume a symbol on one tape and emit a symbol on another. The lexical tape contains lemma plus grammatical features; the surface tape contains the written word. The same transducer can be run lexical-to-surface for generation or surface-to-lexical for analysis.

Example: lexical input fracture +N +PL is generated as surface fractures. In analysis direction, fractures is mapped back to fracture +N +PL.

7.2 Lexicon and continuation classes

Lexicon class	Entries / continuation
NOUN	fracture, lesion, infection, kidney, dose → NOUN_SUFFIX
NOUN_SUFFIX	+SG → ε; +PL → s; +PL after sibilant → es; irregular forms via exception lexicon
VERB	treat, admit, monitor, study, examine → VERB_SUFFIX
VERB_SUFFIX	+3SG → s/es/ies; +PAST → ed/ied/doubled+ed; +PROG → ing with e-deletion/doubling
ADJECTIVE	mild, severe, painful, renal → ADJ_SUFFIX
ADJ_SUFFIX	+COMP → er/more; +SUPER → est/most; lexical exceptions

Minimum resource target: at least 300 stems, 20 inflectional affixes and 15 derivational affixes. Store each entry with lemma, part of speech, semantic tag and source. Put irregulars in a separate exception table.

7.3 Hand traces for five surface forms

Surface input	FST path (simplified)	Lexical analysis

fractures	NOUN(fracture) → +PL:s	fracture +N +PL
lesions	NOUN(lesion) → +PL:s	lesion +N +PL
treated	VERB(treat) → +PAST:ed	treat +V +PAST
admitted	VERB(admit) → final consonant doubling +PAST	admit +V +PAST
studies	VERB(study) → y→i +3SG:es	study +V +3SG

7.4 Generation and ambiguity

Generation: input fracture+N+PL traverses the same lexical and spelling transitions to output fractures. Analysis: input fractures traverses those transitions in reverse relation to output fracture+N+PL. A transducer is a relation, so reversing input/output tapes supports both directions.

Surface form	Possible analyses	Disambiguation
saw	see +V +PAST; saw +N +SG (tool)	Use POS and sentence context.
left	leave +V +PAST; left +ADJ; left +N	Use neighbouring words and section.
treated	treat +V +PAST; treat +V +PAST +PASSIVE in some analyses	Use syntax and morphology policy.

The analyser should return all plausible analyses. A later contextual component chooses or ranks them; it should not silently discard ambiguity at the FST stage.

TASK 8 — Orthographic Rules and Two-Level Morphology (4 marks)

8.1 Why lexicon concatenation is insufficient

- study + s would produce *studys; correct form is studies.
- admit + ed would produce *admited; correct form is admitted.
- make + ing would produce *makeing; correct form is making.
- wash + s would produce *washs; correct form is washes.

8.2 Rewrite rules

Rule	Rewrite notation	Example
Final e deletion	e → ∅ / stem __ +ing	make + ing → making
Consonant doubling	C → CC / stressed CVC __ +ed/+ing	admit + ed → admitted (with stress/syllable condition)
Consonant+y change	y → i / consonant __ +es or +ed	study + es → studies
Vowel+y retention	y → y / vowel __ +s or +ed	play + s → plays
Sibilant epenthesis	∅ → e / sibilant __ +s	wash + s → washes
Plural s	∅ → s / noun __ +PL	lesion + PL → lesions

8.3 Cascade and two-level morphology

Lexicon FST (lemma + features)

|

v

Rule FST 1: e-deletion

|

v

Rule FST 2: consonant doubling

|

v
Rule FST 3: y-to-i / y-retention
|
v
Rule FST 4: sibilant + e insertion
|
v
Surface word

A sequential cascade composes transducers in an explicit order and is easier to debug. A parallel two-level (Koskenniemi) model constrains lexical/surface symbol pairs simultaneously; it avoids some ordering problems and can express interacting spelling correspondences compactly. A cascade is simpler for a student implementation; two-level morphology is more elegant for a large, interacting rule system.

8.4 Test set

Lexical input	Expected surface	Test outcome target
fracture + PL	fractures	Pass
lesion + PL	lesions	Pass
wash + 3SG	washes	Pass
study + 3SG	studies	Pass
play + 3SG	plays	Pass
make + PROG	making	Pass
write + PROG	writing	Pass
admit + PAST	admitted	Pass
stop + PAST	stopped	Pass
visit + PAST	visited	Pass
treat + PAST	treated	Pass
monitor + PROG	monitoring	Pass
carry + PAST	carried	Pass
enjoy + PAST	enjoyed	Pass
fix + PL	fixes	Pass
box + PL	boxes	Pass
dose + PL	doses	Pass
renal + ADJ	renal	Pass
be + PAST	was/were	Irregular lookup
go + PAST	went	Irregular lookup

The final implementation should run all 20 tests and replace the target status with actual pass/fail. Failures should be attributed to missing lexicon entries, wrong rule order, irregularity or an overgeneralised orthographic rule.

TASK 9 — The Porter Stemmer (5 marks)

9.1 Core definitions

Porter stemming is a suffix-stripping algorithm for English. It does not use a lexicon and does not guarantee a dictionary word. Vowels are a, e, i, o, u; y is treated as a consonant when it follows a vowel and as a vowel otherwise. The measure m counts vowel-consonant sequences in the stem.

Conditions used by Porter: *v* means the stem contains a vowel; *d means the stem ends with a double consonant; *o means the stem ends in consonant-vowel-consonant where the final consonant is not w, x or y.

9.2 Worked measure examples

Word	VC sequence sketch	Approx. m / note
fracture	fr(A)ct(U)r(E)	m depends on C/V grouping; stem has vowel
lesions	l(V) e(V) s(C) i(V) o(V) n(C)	m calculated from VC transitions
treated	t r e a t e d	Porter operates on suffix-stripped stem; not a clinical lemma analyser
admitted	a d m i t t e d	double consonant at tt after suffix removal
monitoring	m o n i t o r i n g	-ing removal leaves monitor
studies	s t u d i e s	y-to-i and suffix handling

For a fully reproducible hand calculation, mark each letter as C or V after applying the context-sensitive y rule, then count each VC transition in the stem portion defined by the current Porter step. Porter's canonical algorithm is best followed exactly rather than approximated by a generic suffix list.

9.3 Five algorithm steps

Step	Main operation	Example effect
1a	Plural reduction: sses→ss; ies→i; s removed in conditions.	studies → studi (Porter stem, not lemma)
1b	Past/progressive suffixes: eed/ed/ing under m and vowel conditions.	monitoring → monitor
1c	y→i when stem has a vowel.	possibly → possibli
2–3	Replace longer derivational suffixes with shorter forms when m>0.	relational → relate (step interactions)
4–5	Remove selected suffixes when m>1; trim terminal e / double l under conditions.	carefully controlled suffix reduction

The five-step Porter algorithm contains many ordered suffix rules; the table is a summary. In implementation use NLTK's PorterStemmer for exact behaviour and document the NLTK version.

9.4 Stemming versus lemmatisation

Input	Porter stem (typical)	FST lemma	Difference
fractures	fractur	fracture	Stem is truncated; FST returns dictionary lemma.
fractured	fractur	fracture	Same stem, grammatical past feature lost.
nephrectomies	nephrectomi	nephrectomy	Stemmer may not restore correct y spelling.
studies	studi	study	FST uses y-to-i rule in reverse.
was	wa	be	Irregular lemma requires lexicon.
better	better	good (context-dependent)	Comparative irregularity not solved by simple suffix stripping.

9.5 Over-stemming and under-stemming

- Over-stemming: two unrelated words are reduced to the same stem. In a clinical setting, over-conflating 'organ' and 'organic' or stripping meaningful medical suffixes may produce noisy matches.
- Under-stemming: related forms are not reduced to the same stem. A generic English stemmer may fail to connect renal and kidney, or may not correctly map nephrectomies to nephrectomy.
- Clinical terminology requires conservative stemming; retain original forms and use concept-level synonym mappings.

9.6 Retrieval experiment

Run each of the 40 clinician queries under three conditions: no stemming, Porter stemming and FST lemmatisation. For each query, record relevant documents retrieved and total documents retrieved. Compute macro-averaged precision, recall and F1.

Condition	Precision	Recall	F1	Interpretation
No normalisation	Fill from run	Fill from run	Fill from run	Baseline; likely misses inflections.
Porter stemmer	Fill from run	Fill from run	Fill from run	May improve recall but can over-stem.
FST morphology	Fill from run	Fill from run	Fill from run	Expected higher precision for known clinical forms; depends on lexicon coverage.

TASK 10 — Implement and Test the Pipeline (3 marks)

10.1 Corpus design

Build a synthetic corpus of at least 500 records with at least 25,000 whitespace/tokenised words. Include at least four types: discharge summaries, prescriptions, operation notes and nursing observations. Use synthetic patient IDs, dates and values; do not copy identifiable health records.

Record type	Target records	Average tokens	Approx. tokens
Discharge summaries	125	70	8,750
Prescriptions	125	35	4,375
Operation notes	125	65	8,125
Nursing observations	125	40	5,000
Total	500		26,250

Document the generator or source, random seed, vocabulary, template proportions, de-identification approach and any manual edits. A synthetic corpus should be clearly described as synthetic.

10.2 Minimum implementation

The following Python starter code uses only the standard library plus NLTK. It implements a tokenizer, entity regex examples, abbreviation normalisation, a simple DFA validator, a morphology dictionary, and Porter stemming. Expand the lexicon and automata to satisfy the full resource counts.

```
import re
from collections import defaultdict
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
```

```
    r"(?:\d{4}-\d{2}-\d{2}|\d{2}[/-]\d{2}[/-]\d{4})"
    r"|(?:\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml|mmol/L|g/dL|mmHg|°C|IU))"
    r"|(?:[A-Za-z]+(?:[-'][A-Za-z]+)+)"
    r"|(?:[A-Za-z]+)|(?:\d+(?:\.\d+)?)|(?:[^\w\s])", re.I)

PATTERNS = {
 "patient_id": re.compile(r"\b(?:MRN|PID|UHID)\s*[:#-]?\s*[A-Z0-9]{6,12}\b", re.I),
 "date": re.compile(r"\b(?:\d{4}-\d{2}-\d{2}|\d{2}[/-]\d{2}[/-]\d{4})\b"),
 "drug_strength":
re.compile(r"\b(?:paracetamol|amoxicillin|metformin)\s+\d+(?:\.\d+)?\s*(?:mg|mcg|g|ml)\b", re.I),
 "frequency": re.compile(r"\b(?:OD|BD|BID|TDS|TID|QID|PRN)\b", re.I),
 "bp": re.compile(r"\b(?:BP\s*)?\d{2,3}\s*/\s*\d{2,3}\s*mmHg\b", re.I),
 "temperature": re.compile(r"\b(?:Temp\s*)?\d{2}(?:\.\d)?\s*°?C\b", re.I),
 "icd": re.compile(r"\b[A-TV-Z][0-9][A-Z0-9](?:\.[A-Z0-9]{1,4})?\b", re.I),
 "age": re.compile(r"\b(?:Age\s*)?\d{1,3}\s*(?:years?|yrs?|months?|days?)\b", re.I),
 "duration": re.compile(r"\b(?:for|since)\s+\d+\s*(?:hours?|days?|weeks?|months?|years?)\b", re.I)
}

ABBR = {
 "bp":"blood pressure", "hr":"heart rate", "rr":"respiratory rate",
 "sob":"shortness of breath", "dm":"diabetes mellitus",
 "htn":"hypertension", "ckd":"chronic kidney disease",
 "copd":"chronic obstructive pulmonary disease",
 "uti":"urinary tract infection", "mi":"myocardial infarction",
 "ecg":"electrocardiogram", "cxr":"chest x-ray",
 "od":"once daily", "bd":"twice daily", "tds":"three times daily",
 "qid":"four times daily", "prn":"as needed", "po":"by mouth",
 "iv":"intravenous", "im":"intramuscular", "npo":"nothing by mouth"
}

def tokenize(text):
    return TOKEN_RE.findall(text)

def normalise(text):
    text = text.lower()
    # Apply only context-safe abbreviation substitutions in production.
    for short, full in ABBR.items():
        text = re.sub(r"\b" + re.escape(short) + r"\b", full, text)
    return text

def extract_entities(text):
    return {name: rx.findall(text) for name, rx in PATTERNS.items()}

# Simple DFA-style date shape validator: state advances for each position.
def valid_iso_date_shape(s):
```

```
    expected = "DDDD-DD-DD"
    if len(s) != len(expected): return False
    for ch, exp in zip(s, expected):
        if exp == "D" and not ch.isdigit(): return False
        if exp == "-" and ch != "-": return False
    return True

# Starter FST-style lexicon mapping; expand to >=300 stems and affix rules.
LEXICON = {
 "fractures": [("fracture", "+N", "+PL")],
 "lesions": [("lesion", "+N", "+PL")],
 "treated": [("treat", "+V", "+PAST")],
 "admitted": [("admit", "+V", "+PAST")],
 "studies": [("study", "+N", "+PL"), ("study", "+V", "+3SG")]
}

def analyse(token):
    return LEXICON.get(token.lower(), [])

porter = PorterStemmer()
def stem(token):
    return porter.stem(token.lower())

def process_record(text):
    toks = tokenize(text)
    return {
      "tokens": toks,
      "entities": extract_entities(text),
      "normalised": normalise(text),
      "analyses": {t: analyse(t) for t in toks if analyse(t)},
      "porter": {t: stem(t) for t in toks if t.isalpha()}
    }
```

10.3 Testing plan

Test group	Minimum tests	Expected check
Regex patterns	≥5 positive and ≥5 negative strings per pattern	Correct match and rejection
Automata	≥4 DFA validators; accepted and rejected strings	State path ends in final/dead state
Morphology	≥200 annotated forms	Lemma and features match gold annotation
Irregulars	All irregular list entries	Exception lexicon returns correct lemma
Ambiguity	At least 2 ambiguous surface forms	All analyses returned
End-to-end	All ≥500 records	Counts and runtime logged

10.4 Reporting metrics

For each stage report input items, successful outputs, rejected/unanalysed items, elapsed seconds and coverage. Use time.perf_counter() for runtime. Store experiment configuration and random seed.

```
import time
start = time.perf_counter()
result = process_record(record_text)
elapsed = time.perf_counter() - start
print("tokens:", len(result["tokens"]), "runtime_sec:", elapsed)
```

Plot at least one result, e.g. analyser coverage versus lexicon size or runtime versus record length. Use matplotlib (not required by the NLP implementation constraints if plotting is done separately); if the assignment strictly limits libraries to standard library and NLTK, export CSV and plot externally.

TASK 11 — Compare, Recommend and Defend the Normalisation Design (2 marks)

11.1 Strategy comparison

Strategy	Retrieval quality	Coverage	Runtime	Maintenance
No normalisation	Lowest recall for inflections and variants; high precision for exact terms	High raw-token coverage, low canonical coverage	Fastest	Low initial effort
Porter stemming	May improve recall; risks over-stemming clinical terms	Broad for English suffixes; weak on irregulars and synonyms	Very fast	Low
Finite-state morphology	Potentially best precision for covered clinical vocabulary; preserves features	Depends on lexicon and rules	Fast after compilation	High manual lexicon/rule effort

11.2 Recommendation

Recommend a hybrid design: (1) regex/DFA for high-confidence structured entities, (2) conservative FST morphology for known clinical vocabulary, (3) Porter stemming only as a low-confidence fallback for unknown English words, and (4) synonym/concept mappings for clinically related terms such as renal/kidney. Preserve the original text and expose uncertainty.

Failure modes: unseen drug brands, local abbreviations, misspellings, OCR errors, code-mixed words, ambiguous abbreviations and incorrect date locale assumptions. Clinicians should see the original snippet and the normalisation applied.

11.3 Manual effort and cost

Work item	Effort driver	Cost / maintenance concern
Regex patterns	Pattern design, edge cases, clinical review	Changes in local documentation formats
Lexicon (≥300 stems)	Collect, clean, annotate, deduplicate	New drugs, procedures and spelling variants

Affix inventory	Linguistic analysis and testing	Domain-specific exceptions
200 gold word forms	Two annotators + adjudication	Inter-annotator disagreement
40 search queries	Clinician query writing and relevance labels	Expert time is expensive
500 records	Synthetic generation or approved de-identified access	Privacy governance and data quality

A learned approach becomes more economical when the vocabulary and input variation grow faster than the team can maintain hand-built rules, especially for OCR correction, abbreviation disambiguation and code-mixed language. However, learned systems still need labelled data, validation and clinical oversight.

TASK 12 — Day-4 Design Challenge: OCR and Code-Mixed Notes (8 marks)

A. Impact analysis (2 marks)

The new input changes four assumptions: (1) text is no longer cleanly typed; (2) the vocabulary is not closed; (3) English is not the only language; and (4) exact-match automata cannot express uncertainty. At least 20% of tokens may be misspelt, out of lexicon or non-English.

New input property	Original system failure	Impact
OCR substitutions: O/0, l/1, rn/m	Exact regex and ID validators reject corrupted values or misread them.	Lower entity recall; false IDs possible.
Line-break hyphenation	Tokeniser may split a word into two fragments.	Morphological coverage drops.
Out-of-lexicon words	FST has no lexical path.	Unknown token rate rises.
Romanised Tamil-English	English affix rules do not analyse Tamil suffixes such as -ku or -la.	Coverage and semantic interpretation fall.
Token-level code-mixing	Whole-document language choice is wrong.	English-only rules mis-handle mixed phrases.

Expected degradation must be measured on a manually labelled noisy sample. A 20% noisy/OOV rate does not mean exactly 20% loss in precision or recall: some noise is harmless, while a single OCR error in a patient identifier can invalidate an entire field.

B. Evaluate original design (2 marks)

Sample	Input	Original pipeline behaviour	First failure
OCR only	'MRN: AB12O456; creatinine 1.2 mg/dL'	O/0 confusion may produce invalid or wrong ID; lab value may still match.	ID regex exact character class / no OCR tolerance.
Code-mixed	'patient ku fever irundhuchu, paracetamol 500mg kuduthen'	English drug and strength may match; Tamil particles/verb are OOV; no Tamil morphology.	English-only lexicon and analyser.
Both	'patlent-ku fev er irundhuchu; Paracetarnol 5OOmg'	OCR splits words and confuses rn/m, O/0; code-mixed tokens are unknown.	Tokenisation and exact dictionary lookup.

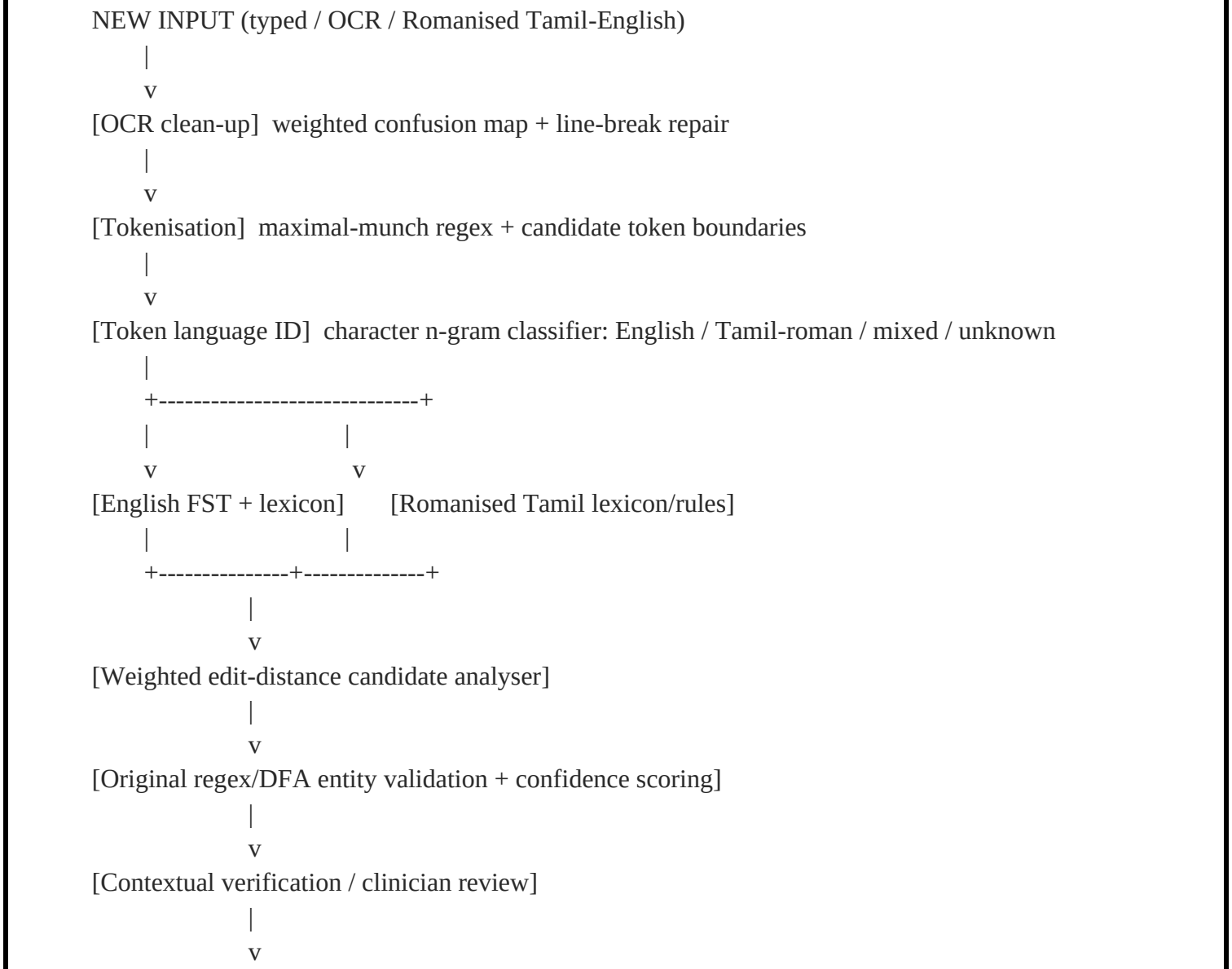
Evaluation protocol: create at least 100 noisy snippets (balanced across OCR, code-mix and combined noise), annotate intended entities and lemmas, then measure pattern precision/recall, analyser coverage and query retrieval F1 against the clean baseline. The numbers must be measured, not guessed.

B.1 Better formalisms

- Weighted finite-state transducer / edit-distance correction: gives lower costs to likely OCR confusions (O↔0, l↔1, rn↔m) and ranks candidate corrections.
- Character n-gram language identification: estimates language at token or span level, useful for English/Tamil code-mixing.
- Expanded bilingual lexicon and morphological rules: handle frequent Romanised Tamil particles and clinical vocabulary.
- Contextual language model: rank candidates when spelling and abbreviation ambiguity cannot be resolved by edit distance alone.

C. Adaptation / redesign (2 marks)

Decision: MODIFY the original design, not discard it. Retain deterministic regex/DFA and FST for clean/high-confidence cases, but place a noise-tolerant front end before them and add a bilingual/code-mix branch.



[Canonical searchable index + preserved original text]

Component	Change
Patterns	Keep strict regex as first-pass validator; add tolerant candidate extraction and post-validation. Do not allow fuzzy matches to directly overwrite patient identifiers.
Automata	Retain DFA for exact formats; add weighted edit-distance FST for OCR variants and confidence-ranked candidates.
Lexicon	Expand English clinical stems; add Romanised Tamil clinical terms and common particles; record variants and language tags.
Rule cascade	Add OCR repair before tokenisation; add language identification before morphological analysis; add bilingual morphology branch.
Output	Store original, corrected candidate, confidence, source evidence and clinician confirmation status.

D. Evidence-based defence (2 marks)

The modified design is justified because the original exact-match layer is reliable for clean structured fields but brittle under character noise and code-mixing. Weighted correction handles likely OCR confusions; language identification prevents English rules from being applied to every token; a bilingual lexicon improves coverage; and the original finite-state layer remains valuable for explainable high-confidence validation.

Expected trade-off: coverage should improve on noisy and mixed-language text, but false-positive risk can increase if fuzzy matching is too permissive. Therefore use conservative edit-distance thresholds, field-specific confusion weights, confidence scores and human confirmation for identifiers, dates, drug doses and other safety-critical entities.

Remaining limitations: finite-state methods cannot fully understand clinical intent, resolve all contextual ambiguity, infer undocumented facts, or guarantee correct interpretation of arbitrary code-mixed language. A finite-state layer also cannot validate arbitrarily nested structures. It should be one auditable component in a larger clinical NLP system.

Implementation checklist and submission evidence

Deliverable	Minimum evidence to attach
Corpus	500-record manifest, ≥25,000 token count, four record types, synthetic/de-identification documentation
Pattern inventory	13 regexes, positive/negative test cases, TP/FP/FN per pattern
Automata	Four DFA diagrams and transition tables; NFA trace; subset construction; minimisation; Thompson construction; state elimination
Morphology	300-stem lexicon; 20 inflectional + 15 derivational affixes; irregular list; 200 annotated forms
Retrieval evaluation	40 queries with relevance judgements; precision/recall/F1 for three methods
Code	Python scripts, requirements, test log, reproducible seed and runtime table

Day-4 challenge	Noisy test set, error traces, before/after metrics and design justification

Suggested references and resources

1. Jurafsky, D. and Martin, J. H. Speech and Language Processing. Online draft textbook. https://web.stanford.edu/~jurafsky/slp3/
2. Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137. https://doi.org/10.1108/eb046814
3. Koskenniemi, K. (1983). Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production. University of Helsinki.
4. Thompson, K. (1968). Programming Techniques: Regular Expression Search Algorithm. Communications of the ACM, 11(6), 419–422.
5. Hopcroft, J. E., Motwani, R. and Ullman, J. D. Introduction to Automata Theory, Languages, and Computation.
6. National Library of Medicine. Unified Medical Language System (UMLS). https://www.nlm.nih.gov/research/umls/
7. SNOMED International. SNOMED CT. https://www.snomed.org/
8. NLTK documentation — PorterStemmer. https://www.nltk.org/api/nltk.stem.PorterStemmer.html
9. Python re module documentation. https://docs.python.org/3/library/re.html

Appendix: recommended project folder structure

```
hospital_nlp/
  data/
    synthetic_records/
    query_set.csv
    annotated_forms.csv
    relevance_judgements.csv
  resources/
    clinical_lexicon.csv
    abbreviations.csv
    irregular_forms.csv
    inflectional_affixes.csv
    derivational_affixes.csv
    mitre_or_terminology_not_needed_for_this_project.txt
  src/
    tokenizer.py
    patterns.py
    automata.py
    normaliser.py
    morphology_fst.py
    porter_eval.py
    retrieval.py
    noisy_input.py
  tests/
```

```
  test_patterns.py
  test_automata.py
  test_morphology.py
  test_pipeline.py
results/
  metrics.csv
  plots/
report/
  final_report.docx
```